

DISCIPLINA: TESTE DE SOFTWARE

Refatoração de Testes e Detecção de Test Smells

Aluno(a): Joao Vitor Pedersoli Rajao

Matrícula: [Matrícula a preencher]

Relatório de análise, refatoração e validação

Outubro de 2026

1. Análise dos Test Smells

O repositório-base contém uma suíte em **test/userService.smelly.test.js**. Antes da alteração, seus quatro testes ativos passavam e um estava ignorado. A leitura manual encontrou os seguintes problemas:

1. Assertivas condicionais e lógica no teste

O teste de desativação percorre usuários com **for** e escolhe expectativas com **if**. Cada execução valida caminhos diferentes no mesmo teste. Uma condição alterada pode impedir que determinada assertiva rode, dando uma falsa sensação de cobertura. O ESLint confirmou três ocorrências de *jest/no-conditional-expect* nesse bloco.

2. Eager Test: responsabilidades em um único caso

O teste “deve criar e buscar um usuário corretamente” valida duas operações, e o teste de desativação cobre usuários comuns e administradores ao mesmo tempo. Quando falham, o nome e a saída não deixam claro qual comportamento regrediu. A versão limpa separa cadastro, busca e cada regra de desativação.

3. Teste de exceção que pode passar silenciosamente

O cadastro de menor está dentro de **try/catch**, mas não há assertiva após o **try**. Se a função deixar de lançar, o teste termina sem executar qualquer **expect** e ainda passa. A versão limpa usa **toThrow**, que falha quando a exceção esperada não ocorre.

4. Fragilidade e teste desativado

O relatório original é verificado por uma linha formatada com ID e pontuação exatos: mudanças cosméticas podem quebrar o teste. Já o cenário de lista vazia estava em **test.skip**, sem proteção ativa. A refatoração verifica informações relevantes do relatório e implementa o caso vazio.

Observação: o ESLint detecta regras sintáticas configuradas; problemas de foco e assertivas frágeis também exigem revisão humana.

2. Processo de refatoração

Exemplo: o teste original de desativação mistura dois cenários em um laço e decide as expectativas em tempo de execução.

Antes — trecho do arquivo original

```
for (const user of todosOsUsuarios) {
  const resultado = userService.deactivateUser(user.id);
  if (!user.isAdmin) {
    expect(resultado).toBe(true);
    const usuarioAtualizado = userService.getUserById(user.id);
    expect(usuarioAtualizado.status).toBe('inativo');
  } else {
    expect(resultado).toBe(false);
  }
}
```

Depois — dois testes focados em Arrange, Act, Assert

```
test('desativa um usuário comum', () => {
  const usuario = userService.createUser('Comum', 'comum@teste.com', 30);
  const desativou = userService.deactivateUser(usuario.id);
  expect(desativou).toBe(true);
  expect(userService.getUserById(usuario.id))
    .toEqual(expect.objectContaining({ status: 'inativo' }));
});

test('mantém ativo um administrador quando a desativação é solicitada', () => {
  const administrador = userService.createUser('Admin', 'admin@teste.com', 40, true);
  const desativou = userService.deactivateUser(administrador.id);
  expect(desativou).toBe(false);
  expect(userService.getUserById(administrador.id))
    .toEqual(expect.objectContaining({ status: 'ativo' }));
});
```

Cada caso prepara apenas seu usuário, executa uma ação e verifica o resultado e o estado observável. Isso remove o **if** e o **for**, torna a falha específica e cobre também a permanência do administrador ativo. Outros cenários limpos exercitam busca inexistente, desativação inexistente, relatório vazio e rejeição de menor.

3. Relatório da ferramenta e validação

Primeira execução de **npx eslint .**, antes de criar o arquivo limpo. A imagem abaixo reproduz a saída capturada em **evidencias/eslint-antes.txt**.



```
$ npx eslint .

/Users/joaorajao/Downloads/TESTE-SOFT/test-smelly/test/userService.smelly.test.js
 44:9  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
 46:9  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
 49:9  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
 73:7  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
 77:3  warning Disabled test                  jest/no-disabled-tests
 77:3  warning Test has no assertions          jest/expect-expect

6 problems (4 errors, 2 warnings)
```

A análise automática apontou quatro erros de assertiva condicional e dois avisos do teste desativado. Ela acelerou a localização dos pontos problemáticos, enquanto a análise manual identificou o Eager Test, a fragilidade da formatação e o falso positivo do try/catch.

Validação final

npx eslint __tests__/_userService.clean.test.js: nenhum erro ou aviso. **npm test -- --runInBand**: duas suítes passaram; 13 testes passaram e o único teste ignorado continua no arquivo original, preservado conforme o enunciado.

4. Conclusão

Testes curtos, com cenários independentes e assertivas obrigatórias, deixam as regressões mais visíveis e reduzem o custo de manutenção. O ESLint oferece uma verificação repetível para certos smells; combinado com revisão humana e execução dos testes, ajuda a manter a suíte confiável ao longo das mudanças no software.

Fontes: repositório-base <https://github.com/CleitonSilvaT/test-smelly>; enunciado da atividade fornecido pelo docente.